1 Implementing Naive Bayes Theorem

[66]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
```

[67]:
```
# Load the dataset
df = pd.read_excel('Naive_Bayes_Dataset.xlsx')  # Adjust path as needed
```

[68]:
```
df
```

[68]:

	Day	Discount	Free_Delivery	Purchase
0	Weekday	Yes	Yes	Yes
1	Weekday	Yes	Yes	Yes
2	Weekday	No	No	No
3	Holiday	Yes	Yes	Yes
4	Weekend	Yes	Yes	Yes
5	Holiday	No	No	No
6	Weekend	Yes	No	Yes
7	Weekday	Yes	Yes	Yes
8	Weekend	Yes	Yes	Yes
9	Holiday	Yes	Yes	Yes
10	Holiday	No	Yes	Yes
11	Holiday	No	No	No
12	Weekend	Yes	Yes	Yes
13	Holiday	Yes	Yes	Yes
14	Holiday	Yes	Yes	Yes
15	Weekday	Yes	Yes	Yes
16	Holiday	No	Yes	Yes
17	Weekday	Yes	No	Yes
18	Weekend	No	No	Yes
19	Weekend	No	Yes	Yes
20	Weekday	Yes	Yes	Yes
21	Weekend	Yes	Yes	No
22	Holiday	No	Yes	Yes

```
23  Weekday      Yes           Yes      Yes
24  Holiday       No            No       No
25  Weekday       No           Yes       No
26  Weekday      Yes           Yes      Yes
27  Weekday      Yes           Yes      Yes
28  Holiday      Yes           Yes      Yes
29  Weekend      Yes           Yes      Yes
```

[69]:
```
# Encode categorical variables
label_encoders = {}
for column in ['Day', 'Discount', 'Free_Delivery', 'Purchase']:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le  # Save encoder for future decoding if needed
```

[70]:
```
# Split data into features (X) and target (y)
X = df[['Day', 'Discount', 'Free_Delivery']]
y = df['Purchase']
```

[71]:
```
# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)
```

[72]:
```
# Initialize and train the Naive Bayes model
nb_model = CategoricalNB()
nb_model.fit(X_train, y_train)
```

[72]:
```
CategoricalNB()
```

[73]:
```
# Make predictions on the test set
y_pred = nb_model.predict(X_test)
```

[74]:
```
# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

           1       1.00      1.00      1.00         6

    accuracy                           1.00         6
   macro avg       1.00      1.00      1.00         6
weighted avg       1.00      1.00      1.00         6
```

[75]:
```
# Example prediction
example = pd.DataFrame({'Day': ['Holiday'], 'Discount': ['Yes'],␣
 ↪'Free_Delivery': ['No']})
for column in example.columns:
    example[column] = label_encoders[column].transform(example[column])
prediction = nb_model.predict(example)
print("Purchase Prediction:", label_encoders['Purchase'].
 ↪inverse_transform(prediction))
```

```
Purchase Prediction: ['Yes']
```

[76]:
```
# Check if "Holiday" exists in the Day encoder classes
print("Classes for Day:", label_encoders['Day'].classes_)
```

```
Classes for Day: ['Holiday' 'Weekday' 'Weekend']
```

[77]:
```
# Example prediction
example = pd.DataFrame({'Day': ['Holiday'], 'Discount': ['Yes'],␣
 ↪'Free_Delivery': ['No']})
# This line creates a DataFrame named example with a single row representing an␣
 ↪example input.

# Encode the example input using updated encoders
for column in example.columns:
    example[column] = label_encoders[column].transform(example[column])
#Purpose: This loop iterates over each column in the example DataFrame and␣
 ↪transforms the categorical
#text values into numeric codes, which the model requires as input.


# Make prediction
prediction = nb_model.predict(example)
#Purpose: This line uses the Naive Bayes model (nb_model) to make a prediction
# on the transformed example input.


# Decode the prediction back to original label
print("Purchase Prediction:", label_encoders['Purchase'].
 ↪inverse_transform(prediction))
```

```
Purchase Prediction: ['Yes']
```

[79]:
```
# Example prediction
example = pd.DataFrame({'Day': ['Weekend'], 'Discount': ['Yes'],␣
 ↪'Free_Delivery': ['Yes']})
# This line creates a DataFrame named example with a single row representing an␣
 ↪example input.
```

```
# Encode the example input using updated encoders
for column in example.columns:
    example[column] = label_encoders[column].transform(example[column])
#Purpose: This loop iterates over each column in the example DataFrame and␣
 ↪transforms the categorical
#text values into numeric codes, which the model requires as input.


# Make prediction
prediction = nb_model.predict(example)
#Purpose: This line uses the Naive Bayes model (nb_model) to make a prediction
# on the transformed example input.


# Decode the prediction back to original label
print("Purchase Prediction:", label_encoders['Purchase'].
 ↪inverse_transform(prediction))
```

```
Purchase Prediction: ['Yes']
```